# SHoEbox CPU profile and training optimization

## Outcome

The current 18-region SHoEbox architecture now trains faster without changing its numerical trajectory. The selected runtime adds an exact feature selector, a derived router-key layout, cached state-read calculations and removal of calculations whose outputs are unused by the current policy.

| Parameters | Reference bytes/s | Optimized bytes/s | Throughput gain |
|---|---|---|---|
| 1M | 14,238 | 19,721 | +38.5% |
| 5M | 12,606 | 19,180 | +52.1% |

These are training-only medians from five repetitions on one pinned CPU. A token is one byte in this experiment. Parameter counts remain 1,023,744 and 4,990,720; region cadence, experts selected, model outputs, learning rates and update order are unchanged.

Additional cores help in two measured ways: running snapshot validation/checkpoint compression beside the trainer, and running independent seeds or controls concurrently. Splitting each tiny expert calculation among two or four CPU workers was slower and is not enabled in the delivered default runtime.

## Where time was going

| 5M component | Reference share | Optimized share |
|---|---|---|
| feature mask | 18.3% | 2.5% |
| routing | 13.8% | 4.8% |
| expert forward | 21.2% | 29.3% |
| expert weight update | 8.0% | 11.2% |
| state publication | 3.9% | 2.3% |

The original top-16 coordinate selector used a general partition algorithm with indirect comparisons. An exact radix selector uses the monotonic bit ordering of nonnegative float32 magnitudes. It preserves lower-index tie breaking, signed-zero behavior and the original NaN fallback. Both AVX512 and AVX2 implementations are supplied.

Router keys are also kept in a derived transposed layout. SIMD lanes score independent experts while preserving the original coordinate accumulation order. Every key update refreshes its derived row, and each native call refreshes the layout so edits made from Python are visible. This adds 54 KiB or 288 KiB of derived cache, not trainable parameters.

The state-read tanh values are reused until their parameters change. Inactive DFA projection work and an unused decoder normalization calculation are skipped. Activation functions and precision are unchanged; fast-math is not used.

The profiling build samples approximately one byte in 64 using a hash of the byte clock, avoiding alignment with regional cadences. Shares are exclusive sampled wall times and include timer overhead. They locate hotspots; all throughput claims come from the separate uninstrumented builds. A larger remaining share does not imply that component became slower.

| Model | Cumulative change | Training bytes/s | Gain |
|---|---|---|---|
| 1M | Reference | 14,238 | +0.0% |
| 1M | Exact feature selector | 18,201 | +27.8% |
| 1M | + router layout | 18,124 | +27.3% |
| 1M | + cached reads / unused work removed | 19,721 | +38.5% |
| 5M | Reference | 12,606 | +0.0% |
| 5M | Exact feature selector | 15,651 | +24.2% |
| 5M | + router layout | 18,269 | +44.9% |
| 5M | + cached reads / unused work removed | 19,180 | +52.1% |

## Using a second core for validation and checkpoints

| Model | Serial seconds | Overlapped seconds | Less elapsed time | Speedup |
|---|---|---|---|---|
| 1M | 38.42 | 25.79 | 32.9% | 1.49x |
| 5M | 42.00 | 26.61 | 36.6% | 1.58x |

Each measurement trains 400,000 bytes with a full 131,072-target validation pass and compressed checkpoint every 100,000 bytes. These are one paired end-to-end trial per model size. Both cases use the optimized training kernel, include all four saves and evaluations, and wait for final completion.

The producer takes a complete independent snapshot after its native training call. A background worker saves that snapshot and validates a frozen copy while the producer resumes training. The queue allows one pending snapshot, so slow validation applies backpressure instead of consuming unbounded memory. Results retain their exact training-target count. Constructor locking protects the inherited native-library binding setup across Python threads.

All four validation scores, all four saved snapshot fingerprints and the final trainer fingerprint match exactly between serial and overlapped execution. The gain depends on how often validation and saving occur; this table uses the current frequent shakedown cadence.

## Multicore training: choose the right work unit

For a single model, parallelizing independent rows inside the 64-wide expert preserves numerical results but introduces a handoff at every regional event. A persistent worker team was used, with workers spread across distinct cache groups and sleeping outside native training calls.

| Model | Kernel / CPU workers | Training bytes/s | CPU-seconds / wall-second |
|---|---|---|---|
| 1M | cached 1 | 19,721 | 1.00 |
| 1M | threaded 1 | 19,018 | 1.00 |
| 1M | threaded 2 | 13,877 | 2.00 |
| 1M | threaded 4 | 8,979 | 3.99 |
| 5M | cached 1 | 19,180 | 1.00 |
| 5M | threaded 1 | 19,322 | 1.00 |
| 5M | threaded 2 | 16,405 | 2.00 |
| 5M | threaded 4 | 11,300 | 3.99 |

The one-worker threaded wrapper controls for the refactoring itself. The two- and four-worker variants are retained only as measured alternatives. The selected default remains the faster one-worker runtime.

Independent trainers are a larger work unit. Each has its own parameters, recurrent state and memory. Native calls release the Python GIL, allowing separate trainers to run concurrently:

| Model | Independent trainers | Aggregate bytes/s | Vs one optimized trainer |
|---|---|---|---|
| 1M | 1 | 17,814 | 1.00x |
| 1M | 2 | 35,933 | 2.02x |
| 1M | 4 | 65,767 | 3.69x |
| 1M | 5 | 79,781 | 4.48x |
| 1M | 8 | 107,629 | 6.04x |
| 5M | 1 | 18,213 | 1.00x |
| 5M | 2 | 36,064 | 1.98x |
| 5M | 4 | 67,979 | 3.73x |
| 5M | 5 | 82,117 | 4.51x |
| 5M | 8 | 130,019 | 7.14x |

Aggregate throughput is the total across separate runs, not the speed of one shared-weight model. Two repetitions used reverse cohort orders, 4,096 warmup bytes and 65,536 measured bytes per worker, with a start barrier. Loading and warmup are excluded. Identical starting checkpoints and byte streams are used here to verify equal numerical results; production cohorts can use different seeds and controls.

## Architecture lock and correctness

The optimized runtime matches the reference bit for bit across 100,000 consecutive training updates for both model sizes. The check includes all trainable arrays, learned state, publication laces, routing statistics, persistent memory and fitted preprocessing means. All paired timing runs also produce identical loss values.

Additional checks cover fresh initialization at the original high learning rate, save/reload continuation, arbitrary native-call chunk boundaries, Python-side key edits, target-independent evaluation, unchanged fitted means during evaluation, and complete snapshot ownership. The AVX2 build matches its reference over 32,768 updates at both sizes. Feature-selector checks compare 12,004 vectors across eight feature counts, including ties, zeros and infinities.

The default native source is byte-for-byte identical to the measured final candidate. Existing version-16 checkpoints load unchanged. Optimized data layouts are derived at runtime and do not add learned weights or checkpoint fields.

Measurements ran on an AMD EPYC 9V74 host with AVX512, nine visible CPUs and a quota of eight CPU equivalents. Cache metadata groups the visible CPUs as 0-1, 2-3, 4-5, 6-7 and 8. Concurrent work was spread across groups first. The raw results contain individual timings, CPU time and all equivalence checks.

## Use for the long training phase

Use the delivered optimized model wrapper for the locked centered-Y/separate-normalization policy. For one active training run, a second CPU worker can perform snapshot validation and checkpointing. For the planned seed/control comparison, run independent one-worker trainers concurrently and budget validation workers within the available CPU quota.

The archive includes the optimized runtime, frozen numerical reference, profiling and benchmark scripts, raw results, correctness checks, the two original 3M checkpoints and the exact profiling data. README_CPU_PROFILE.md contains integration and reproduction examples. SHA256SUMS_CPU_PROFILE.json records the packaged file hashes.

Profiling continuations replayed a fixed data segment to compare speed and numerical identity. They are not additional unique-data training progress. The original checkpoints remain the starting point for the planned longer run.